

Research paper

Multi-agent proximal policy optimization supported dynamic behavior tree evolution for multi-agent systems

Koray Ozdemir^a,*, Adem Tuncer^b^a Dept. of Computer Engineering, Institute of Graduate Studies, Yalova University, Yalova, 77200, Türkiye^b Dept. of Computer Engineering, Engineering Faculty, Yalova University, Yalova, 77200, Türkiye

ARTICLE INFO

Keywords:

Behavior trees
Multi-agent reinforcement learning
Multi-agent proximal policy optimization
Decision-making
Autonomous systems
Dynamic policy evolution

ABSTRACT

In this study, we present a new hybrid approach, Multi-Agent Proximal Policy Optimization (MAPPO)-supported dynamic behavior tree evolution, for improving decentralized decision-making in robotic multi-agent systems. Behavior Trees, despite providing modular and interpretable control structures, are inherently static and may not be able to adapt to dynamic environments. Our method provides scalable and interpretable decision making by allowing agents to independently update their Behavior Tree structures using policy gradients acquired from centralized MAPPO training. We assess the framework within Robot Operating System and Gazebo in three multi-agent scenarios (task allocation, obstacle avoidance, and collaborative execution), and show the superior performance in terms of task success rate, decision latency, structural complexity, adaptation rate, and coordination efficiency with respect to baseline methods. The experimental results demonstrate that the proposed method appropriately strikes a trade-off between the learning-based adaptability and the interpretability of the behavior, which is a hopeful method in the real world multi-agent robotic applications.

1. Introduction

Autonomous systems are a core part of robotic automation, self-driving cars, and multi-agent coordination systems. These systems are fundamentally challenged when making decisions in dynamic and uncertain environments. The use of Finite-State Machines (FSMs) and rule-based systems produces good results for certain tasks, but their scaling problems and their weakness to adapt to the values of the environment have been presented (Colledanchise and Ögren, 2018). FSM suffers from the classical state explosion problem that the more states in the model, the more complex and unrealistic the decision in real time (Lee and Yannakakis, 2002). Rule-based systems require enormous human effort for the acquisition of rules and conditions, rendering themselves inflexible to the different scenarios (Russell and Norvig, 2010).

To address such limitations, Behavior Trees (BTs) are currently considered as a new approach to decision-making for control and cooperative multi-agent system realizations (Ögren, 2012). BTs have a hierarchical, modular, and comprehensive structure that can offer higher reuse potential and debug capability compared with FSMs (Colledanchise and Ögren, 2018). However, the main drawback of BTs is that they are predefined action sequence-dependent, so that their capability to be in accordance with the changeable task environment is limited.

Once a BT is implemented, it remains static unless modified by a human, making it unsuitable for autonomous agents that must acquire and adapt to continuously evolving sets of activities (Marzinotto et al., 2014). This high rigidity severely hinders its applicability to real-world problems in the presence of uncertainty and variability. There have been a few related studies that attempted to improve the flexibility of BTs using learning-based methods that concentrate on Reinforcement Learning (RL)-based structural adaptation mechanisms (Li et al., 2024).

Given the static nature of BTs, researchers have investigated learning-based approaches such as RL to overcome these rigidity issues. RL has, on the other hand, been proposed as a powerful alternative for adaptive decision making, where the agent can learn optimal policies by interacting with its environment (Sutton and Barto, 2018). By exploiting reward-eliciting devices, RL agents continue to optimize their behavior in an iterative manner, and thus are particularly suitable for tasks for which fixed rule systems do not generalize well (Kaelbling et al., 1996). Among RL algorithms, Proximal Policy Optimization (PPO) has shown good performance and stability in robotic tasks, due to the effective policy update procedure (Schulman et al., 2017). Moreover, Soft Actor-Critic (SAC) has been especially effective in continuous action spaces, and is suitable for fine-grained robotic control (Haarnoja

* Corresponding author.

E-mail addresses: 195105003@ogrenci.yalova.edu.tr (K. Ozdemir), adem.tuncer@yalova.edu.tr (A. Tuncer).

et al., 2018). Interpretable BTs have also been learned from RL policies for transparency and human-understandability in recent studies (Zhao et al., 2023).

However, most RL algorithms operate with a single-agent approach and thus are not suitable for multi-agent environments, where agents must coordinate, share tasks, and compete (Busoniu et al., 2010). In our previous study, a reinforcement learning-based approach has been introduced for autonomous mobile robots to navigate dynamic and unknown environments using dueling double deep Q-networks (Ozdemir and Tuncer, 2025). This study focused on enabling a single robot to autonomously learn optimal policies for navigation in dynamic environments. The presence of multiple learning agents introduces non-stationary dynamics where the learning process of agents leads the environment to keep changing, and it becomes hard for the traditional RL methods to learn optimal policies (Hernandez-Leal et al., 2019). This limitation has resulted in the emergence of Multi-Agent Reinforcement Learning (MARL), which represents the interactions of agents explicitly and optimizes policies jointly, rather than independently (Lowe et al., 2017). MARL has been shown to be effective in robotic swarms, autonomous vehicle groups, and distributed sensor networks, which proved that MARL holds the capacity to improve cooperation and competition in agents' decision making (Zhang et al., 2021). Related frameworks such as LGC-MARL have shown that hierarchical decomposition and modular structure enhance MARL efficiency in dynamic environments (Jia et al., 2025).

One of the most relevant MARL techniques is known as Multi-Agent Proximal Policy Optimization (MAPPO), a variant of PPO and a framework for centralized training with decentralized execution (Zhang et al., 2024). MAPPO supports agents to learn shared policies with decentralized decision making, which is a scalable and computationally effective approach for large-scale multi-agent systems (Iqbal and Sha, 2019). It has been demonstrated that MAPPO yields better allocation of tasks, coordination among tasks, and system performance than the others, especially in dynamic and adversary environments (Sadhukhan and Selmic, 2021). Recent studies have even introduced intent sharing as a way to improve coordination in cooperative decision-making in connected autonomous vehicle systems (Guo et al., 2025). Nonetheless, MAPPO and other MARL algorithms suffer from the lack of a structured and interpretable reasoning framework for decision logic representation, which makes encoding and debugging complex hierarchical behaviors challenging.

Despite their strength in adapting to changes in the environment, many RL-based techniques have difficulty being interpreted, modularized, and human-in-the-loop tunable (Garnelo and Shanahan, 2019). In contrast to RL, BTs offer interpretability and modularity but lack adaptive learning capabilities (Colledanchise and Ögren, 2018). Therefore, a gap exists in this research area: enabling agents to automatically evolve and adjust their decision structures on the fly in response to dynamic environments using RL remains an open challenge that should be addressed. A line of research concerns the incorporation of RL in BTs for adaptation driven by data. Zhao et al. (2023) introduced a methodology of extracting behavior trees from trained RL policies in order to make them more interpretable. Other methods utilize RL to adjust action parameters inside the framework of a given BT or to choose a sub-tree from a set of pre-defined ones. Although they enhance BT's reactivity, they generally consider the tree as static, and hence they do not benefit from the adaptation aspect. More recently, techniques have evolved action nodes in BTs using genetic programming in order to maximize the performance of agents in tasks simulated by the environment, but these approaches are not efficient and are often not adaptable to real-time multi-agent scenarios (Iovino et al., 2023). Scherf et al. (2023) proposed an interactive method for better BT refinement from human demonstrations (ILBERT), but the lack of autonomous policy-driven structural learning remains. In addition, the MARL-BT model proposed by Li et al. (2024) inlays MARL onto fixed BTs with action masking to

instantaneous deviations, but not to the dynamic structure change of the tree.

To this end, we propose in this study a hybrid framework that combines MAPPO with dynamic Behavior Tree Evolution (BTE) to allow agents to adapt their decision structures during problem solving with learned experiences. Agents can learn the behavior tree nodes individually through decentralized decision-making, while benefiting from centralized training. We empirically evaluate the framework in realistic ROS2 and Gazebo multi-agent robotic settings that use the task success rate, decision latency, structure complexity, and cooperative efficiency as key metrics. This integration not only increases flexibility and modularity but also improves transparency and scalability, providing a practical solution for real-world multi-agent robotics.

The proposed method includes the following contributions:

- We present a hybrid method that combines MAPPO with dynamic BT evolution for improved decentralized decision-making for systems of multi-agent robots.
- We refine static BT models by permitting policies learned using MAPPO to auto-adjust the BT structure in response to changes in the environment.
- Each agent has the capability to update its own BT structure and take local decisions; hence, no control unit is required at the center of the system, and the global complexity of the architecture is scalable.
- BTs optimized in MAPPO facilitate efficient task propagation and collaboration among agents, leading to a reduction in the system's overall task completion time.
- The methodology was developed over a simulated robot system in ROS2 and Gazebo, and its evaluation took place over test scenarios of mobile and industrial robotics.
- We combined MAPPO and the modularity of BTs. So our approach provides a flexible and scalable way to improve autonomous decision-making for complex multi-agent systems.

2. Background

2.1. Behavior Trees (BT)

BTs are becoming more used in the decision-making process in robotics due to their modular and hierarchical organization of decision-making. Every node in the tree represents either an action or a condition; this means that, based on the current state, traversal occurs from the root to the leaf node of the tree. The main benefit of BTs is the interpretability and the debugging, and they are particularly well-suited to autonomous or intelligent systems. Colledanchise and Ögren (2018).

The performance of a BT is typically evaluated through a reward function $R(BT)$, which can be defined as:

$$R(BT) = \sum_{i=1}^n w_i \cdot r_i \quad (1)$$

where w_i represents the weight associated with each action r_i in the tree, and n is the total number of actions. However, one of the main limitations of traditional BTs is their static nature—once a BT structure is defined, it does not adapt to environmental changes without human intervention (Ögren, 2012).

2.2. Reinforcement learning and MAPPO

RL is a widely known machine learning method where agents gain decision-making capability through exploration of the environment and provide feedback in the form of rewards. An RL agent seeks to maximize its expectation for the cumulative reward, represented as:

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R_t \right] \quad (2)$$

where π is the policy, γ is the discount factor, and R_t is the reward at time t (Sutton and Barto, 2018).

In the context of multi-agent, the difficulty is non-stationarity due to the interactions between agents. To solve this problem, MAPPO (Zhang et al., 2024) has been presented as an efficient approach. MAPPO generalizes the traditional PPO algorithm for multi-agent systems by enabling centralized training and decentralized execution. Policy optimization in MAPPO is determined by:

$$L^{\text{MAPPO}}(\theta) = \mathbb{E} [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)] \quad (3)$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio between the new and old policies, A_t is the advantage function, and ϵ is a clipping parameter (Schulman et al., 2017). The MAPPO framework improves task allocation, coordination, and performance in dynamic environments by allowing agents to learn shared policies while executing decentralized actions.

3. Materials and methods

3.1. General approach

We adopted MAPPO and integrated it with BTs to improve and smooth the decisions of a multi-agent autonomous system. BTs are typical decision-making systems, which are modular and can be easily understood by robots, even though they are unsuitable for environmental changes. The role of RL is to let agents learn the optimal policy through environment interactions; however, the decisions made by agents are invisible and haphazard. This study aims to fill the above gap that systems can adjust their BT structures through learning from experience without losing interpretability. The agents could improve the performance of task allocation and other cooperative actions, under environmental adaptation by tuning the structure of the BTs dynamically opposed to using fixed predefined structures. BTs give an opportunity to the agents to change their decision hierarchy when circumstances change. Agents in an integrated MAPPO-BTs system form an innovation decision-making framework that can constantly enhance itself using learning action decision and environmental feedback-based decision logic reconfiguration. We compared our approach to traditional RL-based methods that rely on neural networks, as it retains the structured nature of BTs to produce modular and interpretable learned behaviors.

3.2. System overview

The design of the system's architecture comprises a collection of independent agents whose controller is given by BT, which is adapted dynamically via RL. The architecture enables agents to self-adapt the way they make decisions when they are interacting with the environment. The system agents use MAPPO to arrange and improve their BTs rather than the static BTs, which offers the adaptability and scalability for multi-agent coordination tasks. The system adopts a two-stage execution model: (1) centralized training and (2) decentralized deployment. The critic is centralized with regard to providing feedback on the global performance of the agents during their training, but each agent learns its own policy network, which dictates the actions. The critic provides guidance for policy optimization to make agents learn personal strategies and cooperative behaviors that help improve system efficiency. During execution, agents act independently according to their learned policies and online update their individual BT structures by receiving environmental feedback. This architecture is based on BTs, which is the core feature that enables the agents to evolve their BT structures based on RL feedback. Our methodology thus allows the agents to evolve their decision-making hierarchy through the addition and deletion of BT nodes and node modifications, which sets it apart from standard static BT systems. Integrating MAPPO with BTs, we enable agents to learn how to act according to the shortest horizon

that maximizes their immediate reward, and to exploit decision structure over the long term, which makes the decision execution process more efficient and robust under the dynamic context. The structure enables the design of moving agents capable of learning modular, self-improving decision-making in an understandable manner. So we keep the entire system applicable to various complex multi-agent systems, including collective robotic fleets and autonomous search-and-rescue missions, and industrial control systems.

3.2.1. Algorithmic overview of the proposed method

Algorithm 1 details the training loop in which MAPPO updates and BT structural edits are executed within the same cycle, while Fig. 1 summarizes the end-to-end pipeline. Lines 9–21 of the algorithm highlight how MAPPO updates and BT edits interact: updates are performed using shared advantages, and structural changes (Insert, Prune, Reorder) are triggered by performance, latency, or entropy criteria. These edits are validated and safeguarded with rollback checks to ensure stability. Together, the algorithm and pipeline illustration clarify how structural adaptation is integrated into RL training, consistent with the qualitative literature comparison in Table 9.

Algorithm 1 MAPPO with Structural BT Evolution during RL Training (MAPPO-BTE)

Input: Environment $\mathcal{E}$; initial BTs $\{T_i\}_{i=1}^N$; shared policy π_θ , value V_ϕ ; horizon T ; update period K ; triggers C ; edit ops {INSERT, PRUNE, REORDER} with probs p_I, p_P, p_R .
Output: Evolved BTs $\{T_i^*\}$ and trained (θ^*, ϕ^*) .

```

1: Initialize  $\theta, \phi$ ; empty buffers  $\{B_i\}$ .
2: for episode = 1, 2, ... do
3:   Reset  $\mathcal{E}$ ; get  $\{o_i^0\}_{i=1}^N$ .
4:   for  $t = 0, \dots, T-1$  do
5:     for each agent  $i$  (parallel) do
6:       Traverse BT  $T_i$  to select leaf/template.
7:       Sample  $a_i^t \sim \pi_\theta(\cdot | o_i^t, \text{BT\_ctx}(T_i))$ .
8:     end for
9:     Step env:  $(o^{t+1}, r^t, \text{done}) \leftarrow \mathcal{E}(a^t)$ ; push tuples to  $B_i$ .
10:    if  $(t+1) \bmod K = 0$  or done then
11:      MAPPO update: compute GAE  $\hat{A}_t$ , returns; update  $(\theta, \phi)$  via clipped PPO
        objective (shared advantages).
12:      if trigger  $C$  holds (e.g., low TSR/high DL/low entropy, BT stalls) then
13:        BT evolution: sample  $o \sim \{p_I, p_P, p_R\}$ ; apply to  $T_i$ ;
14:        INSERT new node; PRUNE low-utility subtree; REORDER children by return.
15:        Validate BT (arity/termination); warm-start params; if performance  $\downarrow > \delta$ 
          over  $M$  episodes, rollback.
16:      end if
17:      Clear  $\{B_i\}$  and continue.
18:    end if
19:  if done then
20:    break
21:  end if
22: end for
23: end for

```

3.3. Action space definition

In this study, each agent's action space is characterized as a discrete action space made up of angular and linear velocities. The basic parameters for managing the movement of mobile robots are these velocities. The discrete action space consists of the following pairs of linear velocity (v) and angular velocity (ω):

Linear Velocities (v):

$$v \in \{0, 0.1, 0.2, 0.3, 0.4\} \text{ m/s} \quad (4)$$

where $v = 0$ represents no movement (the agent is stationary), and higher values correspond to faster forward motion.

Angular Velocities (ω):

$$\omega \in \{-\frac{\pi}{4}, 0, \frac{\pi}{4}\} \text{ rad/s} \quad (5)$$

where $\omega = 0$ means no turning (straight movement), $\omega = \pm\pi/4$ corresponds to a turning rate in either direction (left or right).

Thus, the action space consists of a total of 15 discrete actions (i.e., $5 \times 3 = 15$ combinations of linear and angular velocities). This

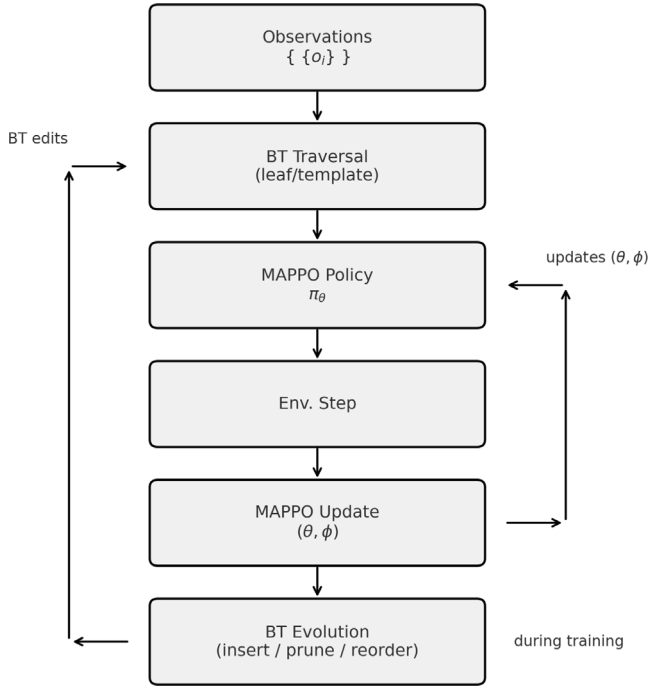


Fig. 1. Pipeline of the MAPPO-BTE framework.

allows the agents to select one of these 15 actions at each time step based on their current state and the learned policy.

The agents in our system select an action from this action space at each decision point based on their current state and the policy learned through MAPPO. The action space is discrete and dynamic, meaning that agents can adapt their action choices as they learn from interactions with the environment.

3.4. Behavior tree evolution mechanism

A key feature of the proposed methodology is the dynamic evolution of BTs, where agents can adapt their decision-making structures through RL. Differently from regular BTs, in which the execution sequences are fixed, our approach allows individual agents to perform structural changes to their own BTs to improve the adaptability and performance in a multi-agent system.

3.4.1. Dynamic modification of BTs

In the static hierarchy of traditional BTs the execution sequence is pre-defined and changeable only by user intervention. Such rigidity hampers the ability of an autonomous agent to adapt to changing environmental conditions. To overcome this limitation, we propose a learning-based BT modification mechanism with three levels:

- **Node Insertion** – A new node N_{new} is added if the policy network detects an action or condition that improves long-term reward.
- **Branch Pruning** – An existing node N_i is pruned if its contribution to the overall reward function $R(BT)$ falls below a threshold.
- **Sequence Reordering** – Nodes are adaptively placed with respect to their overall acceptance ratio and running efficiency.

The modification process is governed by a policy function π_θ trained using MAPPO. Given the current state s_t , the agent selects an action a_t while simultaneously updating its BT structure through a structural transformation function T :

$$BT_{t+1} = T(BT_t, \pi_\theta(s_t)) \quad (6)$$

For readability, only the main objective functions are reported here, while the detailed derivations of probability functions, reward components, parameter settings, and exploration–exploitation balance have been moved to [Appendix](#).

3.4.2. Evaluation and reward function

To evaluate the effectiveness of modifications, we define a reward function $R(BT)$ composed of three components:

$$R(BT) = w_s R_s + w_e R_e + w_a R_a \quad (7)$$

These components reflect Task Success Rate (R_s), Execution Efficiency (R_e), and Adaptability (R_a). The weighting factors are adapted during training to emphasize exploration in the early stage and stability in the later stage.

3.4.3. Structural constraints and stability mechanisms

To avoid over-modifications, structural restrictions are imposed:

- **Tree Depth Limitation:** Maximum depth constrained by D_{max} .
- **Node Pruning Threshold:** Nodes with very low execution probability are removed.
- **Modification Cooldown:** A cooldown period is applied after each major structural change to allow stabilization.

To make the three structural operators more concrete, we explicitly define their selection probabilities and stability checks. Let ΔA denote the recent change in the averaged advantage of an agent over a moving window of episodes. The probabilities of applying INSERT, PRUNE, and REORDER are given by:

$$p_I = \sigma(\lambda_I \Delta A), \quad p_P = \sigma(\lambda_P (1 - \Delta A)), \quad p_R = 1 - (p_I + p_P), \quad (8)$$

where $\sigma(\cdot)$ denotes the logistic function, and λ_I and λ_P control the sensitivity of structural changes to performance variations. After a candidate edit T is applied and evaluated over a short horizon, its effect on the BT-level reward is computed as $\Delta R(BT) = R(BT_{new}) - R(BT_{old})$. A modification is *accepted* when $\Delta R(BT) \geq 0$, and *rejected with rollback* when $\Delta R(BT) < -\delta$, where δ is a small tolerance threshold. All updated BTs are additionally validated against structural constraints (such as acyclicity, maximum depth D_{max} , and the presence of at least one terminating leaf) to prevent degenerate or non-terminating trees.

For clarity, the structural hyperparameters used throughout the experiments are reported below.

In all experiments, we fixed $\lambda_I = \lambda_P = 3.0$, $\tau = 0.1$, $\delta = 0.05$, and $\eta = 0.02$. Here, λ_I and λ_P control operator sensitivities, τ is the triggering threshold described in [Appendix](#), δ determines the acceptance/rollback tolerance, and η is the step size governing updates to the internal confidence score of the transformation policy. These values were selected empirically to balance stability and adaptation speed and were kept constant across all scenarios.

3.4.4. Final selection of BT modifications

In each update step, candidate BT updates are evaluated, and the modification with the highest expected benefit is selected:

$$T^* = \arg \max_{T_i} \Delta R(BT_i) \quad (9)$$

This ensures agents gradually refine their BTs, converging to adaptive, interpretable, and optimized decision structures over training.

3.5. Integration with MAPPO

By using the MAPPO integrated with BTE, the agents can reconfigure their decision hierarchies in a flexible manner and optimize both short-term actions and long-term structural adaptation. Our method therefore extends conventional RL approaches by enabling MAPPO not only to optimize policy execution, but also to dynamically shape the structure of BTs during training.

Table 1
Simulation environment parameters.

Parameter	Value
Number of agents	3, 5, 8
Simulation area	20 m × 20 m
Obstacle density	10%–30%
Number of goals	5
Task types	Navigation, Dynamic obstacle avoidance, Collaborative
Agent type	Differential-drive mobile robot
Sensors	LIDAR, Range sensors

3.5.1. Reinforcement learning-guided BT modification

Each agent follows a *dual-policy approach*: one policy governs short-term action selection, while another determines whether a modification should be applied to its BT. At each step, the agent chooses both an action and, if beneficial, a structural update (e.g., node insertion, pruning, or reordering). This ensures that MAPPO influences not only low-level behavior but also the long-term decision logic encoded in BTs. The underlying probabilistic formulation of this mechanism is provided in [Appendix](#).

3.5.2. Structural transformation selection

The decision to apply structural changes is guided by an adaptive confidence score that increases with successful transformations. A balance between exploration and exploitation is maintained, so that early training emphasizes structural experimentation while later training consolidates stable hierarchies. Mathematical details of the confidence update and regularization are given in [Appendix](#).

3.6. Optimization strategies

3.6.1. Structural optimization of BTs

To avoid unnecessary complexity, redundant nodes are pruned and tree depth is limited. If the BT grows beyond a threshold, underperforming branches are merged or replaced. These mechanisms ensure that BTs remain compact and efficient while still supporting adaptability. Formal definitions of pruning and balancing metrics are provided in [Appendix](#).

3.6.2. Policy optimization in MAPPO

MAPPO training is further stabilized by adopting strategies such as trust-region style updates, centralized critics with decentralized execution, and adaptive learning rates. These techniques collectively mitigate common MARL challenges such as non-stationarity, credit assignment, and sample inefficiency, ensuring robust learning when integrated with evolving BTs. Detailed mathematical formulations of these strategies are deferred to [Appendix](#).

3.6.3. Experimental setup

A distributed environment for multi-agent simulation using ROS2 and Gazebo is built for the validation of the MAPPO integrated BTE framework. The objective is to study how the agents can adapt their action policies and hierarchical decision structure on-line in dynamic and cooperative task environment.

Simulation environment and hardware. We simulated in the Gazebo 11 with ROS2 Foxy-Fitzroy middleware environment. Each agent was simulated as a differential-drive mobile robot with LIDAR and range sensors for sensing the environment. Both training and simulation were performed in a workstation with an AMD Ryzen 9 5950 X CPU, an NVIDIA RTX 3090 GPU, 64 GB memory (RAM) and Ubuntu 20.04.

The simulation parameters used throughout the experiments are summarized in [Table 1](#).

Table 2
MAPPO training hyperparameters.

Hyperparameter	Value
Learning rate	3×10^{-4}
Discount factor (γ)	0.99
GAE lambda	0.95
Batch size	2048
Number of epochs	10
Policy/Critic ratio	1:1
Entropy coefficient	0.01
Training episodes	10,000
Episode length	500 steps

Behavior tree initialization and evolution. Agents began training with the same static BT at the start. The tree structure originated with a root selector node, sequenced a number of sequence nodes and five leaf-level testing action/conditions. The MAPPO algorithm supplied each individual agent with sent-by-policy updates to autonomously grow its own BT structure in the course of learning.

The BT structure underwent significant modifications to fit facts. The agents grow roughly 3.4 new nodes with 2.1 redundant or ineffective ones removed during the process. Depth of BT structures on average deepened from starting average depth of 3 to around 5.2 after training. The structural alterations yielded better flexibility and performance results under different simulation situations.

Training configuration for MAPPO. The MAPPO algorithm was trained centrally and each agent executed its policy in a decentralized way. The main hyperparameters applied all over the training are listed in [Table 2](#).

Training section has 500 steps. The learning stage was repeated for a 10,000 episodes. At the end of each section, the agents are permitted to examine their current BT structure and the impact of changes on performance. This decisions are taken with the aid of the special reward mechanism that is embedded in the BT development.

Scenario definitions. Three experimental scenario were designed in order to provide the generalization of the method. The first scenario is a decentralized task allocation in which the agent selects the goal to minimize task overlap and collision. The second scenario introduced dynamic obstacles in the simulations and the agents needed to be capable of modifying the way they navigate in real time, by means of dynamic obstacle avoidance. The third was collaborative sub-tasks only solvable by joint actions of more than one agent, like collectively defending a common target. The environments for these scenarios are shown in ([Fig. 2](#)), ([Fig. 3](#)) and ([Fig. 4](#)).

As shown in the figures, each scenario presents unique challenges that test the adaptability and coordination capabilities of the agents. All scenarios were tested in 20 repeated trials and the results were evaluated according to the task success rate, decision time and the general structural complexity of the evolved BTs of the agents. The evaluation scenarios used for testing were identical to those employed during training, allowing us to rigorously assess the final performance and convergence of the learned policies in familiar environments. Future studies will investigate generalization to unseen configurations to further validate adaptability.

3.6.4. Evaluation metrics

Several performance evaluation benchmarks were also made to compare MAPPO guided BTE framework in a multi-agent context. The evaluation metrics study the behavior at both the agent and system level. Results were based on 20 repeated trials' averaged performance across agents unless otherwise stated.

The *Task Success Rate (TSR)* is the main evaluation metric used to calculate how well agents successfully completed the task over multiple episodes. The BT structural assessment relies on both its decision effectiveness and its ability to adapt. Solving an episode is defined

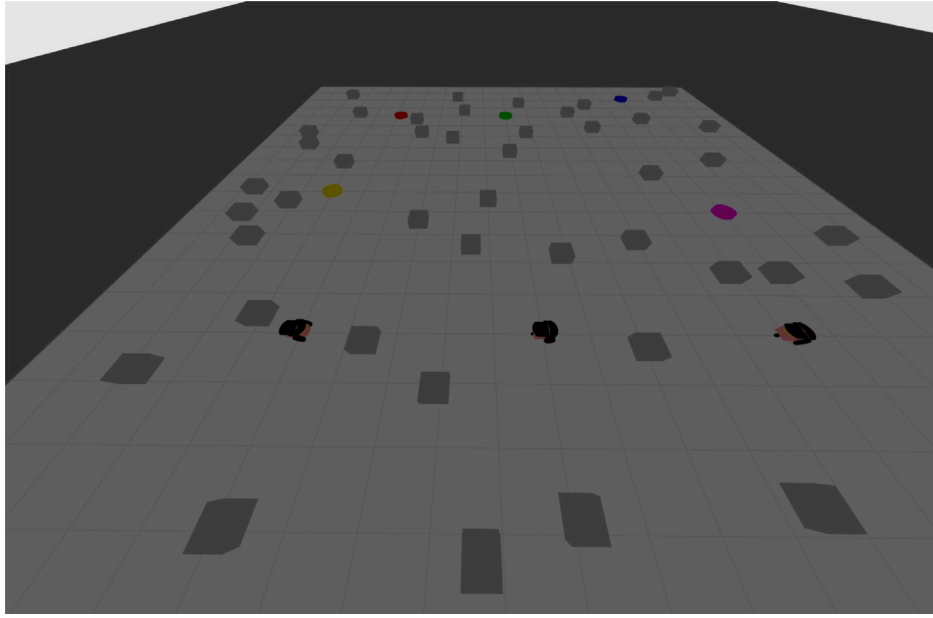


Fig. 2. Task allocation scenario.

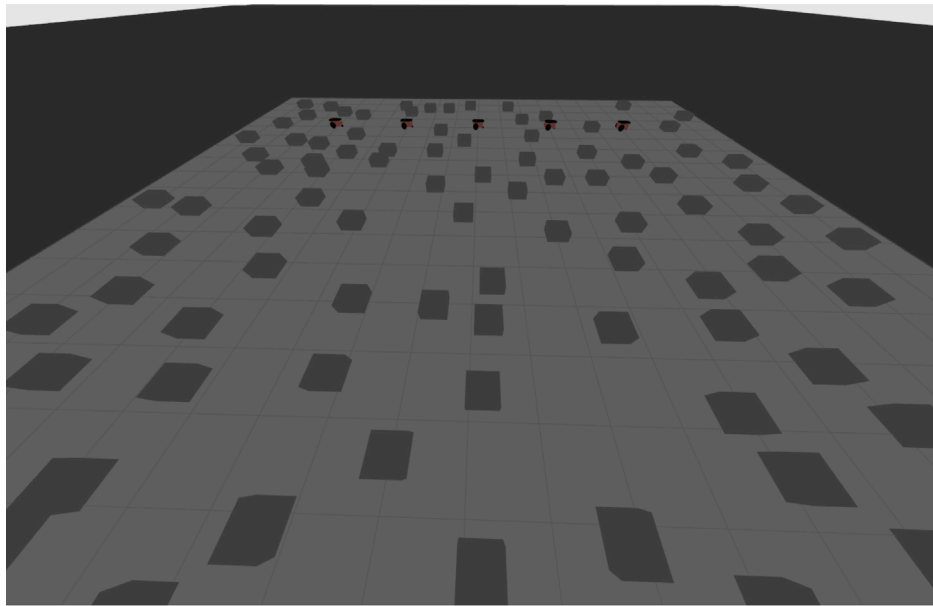


Fig. 3. Dynamic obstacle avoidance scenario.

as agents reaching all destinations before time runs out and without encountering any catastrophic collisions.

The *Decision Latency (DL)* is defined as the average of time steps that it takes for the agents to finally reach their decisions which launching actions respectively. The complication of the BT structure is the main factor to affect this measurement. If the training is good, a good structural optimization would result in DL decreasing or being stable.

The *Behavior Tree Complexity (BTC)* metric monitored the evolved BT structure by counting the number of BT nodes and the average depth of the trees of all agents. This measure plays a pivotal role in the analysis of BT growth behavior to evaluate whether the tree becomes too intricate or remains efficient. A good system exhibits good task execution with modular and shallow trees.

A novel metric, here named *Structural Adaptation Rate (SAR)*, was used to quantify how frequently agents change BT structure and how much it adapts during training. A large initial SAR score indicates that

exploration is very active, but the score should not decrease too much over time as policies converge and BT stabilization occurs.

Besides, the *Multi-Agent Coordination Score (MACS)* was used as an additional measure to assess collaborative scenarios, since it computed the degree of agent synchronicity in joint task performance. Computation of this metric requires identifying the spatial configurations and temporal sequences of agent arrangements, which effectively enables them to achieve objectives collectively. The set of metrics permits the comprehensive evaluation of the individual system as well as scalability, flexibility, and coordination efficiency in behavior-based multi-agent systems.

To further clarify the rationale behind the selected evaluation metrics, we note that each metric targets a specific dimension of the adaptability-interpretability spectrum. TSR quantifies overall performance and thus reflects the system's adaptability to diverse task conditions. DL measures the average time between observation and action

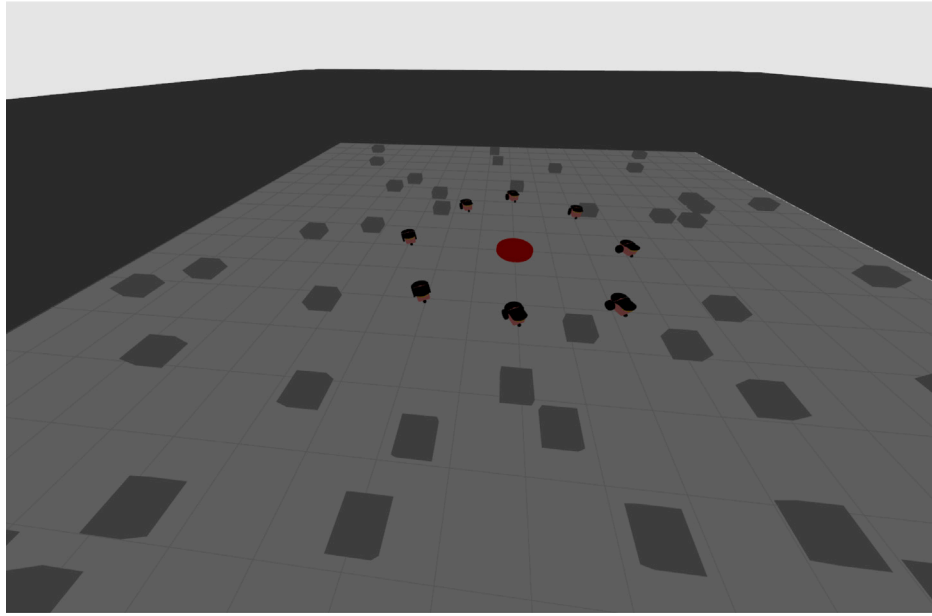


Fig. 4. Collaborative tasks scenario.

Table 3

Final Task Success Rate (TSR) across scenarios.

Scenario	Task success rate
Task allocation	$91.3 \pm 2.1\%$
Dynamic obstacle avoidance	$87.8 \pm 2.4\%$
Collaborative tasks	$83.4 \pm 2.8\%$

Table 4

Average Decision Latency (DL) before and after training.

Scenario	Initial DL (Steps)	Final DL (Steps)
Task allocation	18.2 ± 1.1	11.6 ± 0.9
Dynamic obstacle avoidance	20.4 ± 1.2	13.1 ± 1.0
Collaborative tasks	22.5 ± 1.3	14.7 ± 1.2

execution, capturing the computational efficiency and responsiveness of the evolved BTs. BTC expresses the structural simplicity of the tree (e.g., node count or average depth) and directly correlates with interpretability and ease of debugging. SAR denotes how frequently BT modifications occur during training, serving as a proxy for the flexibility and plasticity of the learning process. Finally, MACS evaluates coordination quality across agents, which links adaptability to the emergent collective behavior rather than individual performance. Together, these complementary metrics jointly capture both the dynamic adaptation capacity and the transparency of evolved multi-agent BTs, aligning with the central objective of balancing scalability, adaptability, and interpretability in MAPPO-BTE.

4. Results and analysis

The experimental results on the proposed MAPPO-guided BTE architecture testing were presented in this section for different multi-agent simulations. The evaluation appraises both individual agent behavior and global coordination by comparing the five evaluation metrics, namely Task Success Rate (TSR), Decision Latency (DL), Behavior Tree Complexity (BTC), Structural Adaptation Rate (SAR), and Multi-agent Coordination Score (MACS). Results are over 20 runs per scenario, and values are averaged over agents unless otherwise stated. The quantitative results at the end of the training process are shown in Tables 3–7.

Table 5

Behavior tree complexity (Node count and depth).

Scenario	Avg. node count	Avg. depth
Task allocation	12.1 ± 1.6	5.2 ± 0.4
Dynamic obstacle avoidance	14.3 ± 1.8	5.6 ± 0.5
Collaborative tasks	15.2 ± 2.0	5.9 ± 0.6

Table 6

Structural Adaptation Rate (SAR) over training.

Scenario	Initial SAR (changes/ep.)	Final SAR (changes/ep.)
Task allocation	2.8 ± 0.3	0.4 ± 0.1
Dynamic obstacle avoidance	3.1 ± 0.4	0.5 ± 0.1
Collaborative tasks	3.5 ± 0.4	0.6 ± 0.1

Table 7

Multi-Agent Coordination Score (MACS) in collaborative tasks.

Scenario	MACS (Initial → Final)
Collaborative tasks	$0.51 \rightarrow 0.78$

4.1. Discussion of results

Combined with the quantitative results in Tables 3–7, our findings collectively verify the efficacy of our MAPPO-supported BTE framework in dynamic, multi-agent scenarios. The findings indicate that the reinforcement learning with modularity, evolvability of decision making of dynamic deciders is not only efficient in some tasks but also versatile in its structure and adaptable to system level coordination.

Table 3 show significant gains on task success in all test cases. The TSR continued to rise during the training stage, which demonstrated the learning process well and the learned structure was useful (Fig. 5). The task allocation scenario obtained a TSR of 91.3% which proved that agents that are not centralized were able to decide and execute the target goals with low-level conflicts. Our framework can operate 87.8% TSR in dynamic environment avoidance and 83.4% in collaboration. The results show that combining adaptive control with modular planning in a hybrid decision-learning model is successful.

Results presented in Table 4 demonstrate that there is an overall trend of decreasing in Decision Latency (DL) and agents are able to traverse the BT structures more efficiently. For example, in the DSO,

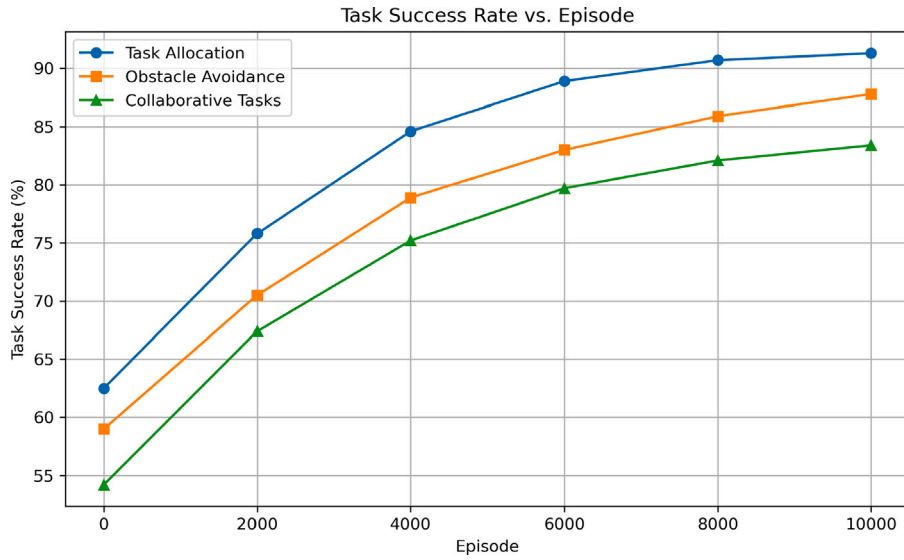


Fig. 5. Task success rate progression across episodes for different task scenarios.

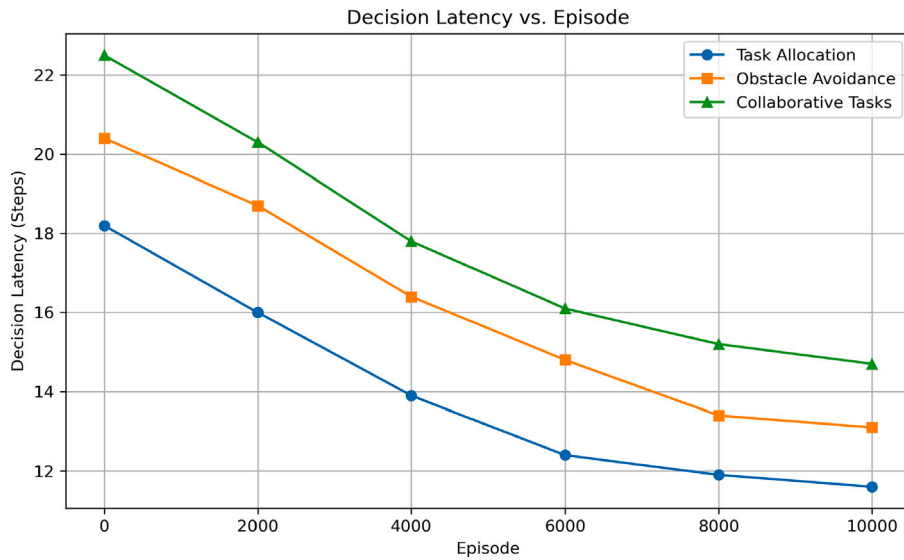


Fig. 6. Average decision latency over episodes.

DL decreased from 20.4 steps to 13.1 steps. This indicates that the agents not only learned better policies, but also redesigned the behavior trees to be shallower and more level. These changes act to influence the responsiveness of the system which, in cases of real-time robotic systems, is of paramount importance as latency can be understood as actuation delay. As shown in Fig. 6, the decision latency always declined throughout episodes, which suggests that the agents learned how to make their BT traversals more efficient with experience.

The average BT complexity (Table 5) increased at the beginning of learning but it reached reasonable depth and node size after a few learning instances. This corresponds to a trade-off between expressive power of a decision and the efficiency of its execution. Agents grew their BTs in a way that better supported environmental diversity, while also learning to prune away ineffective parts of the space. It supports the idea of smart modularization and that agents construct optimum control hierarchies, as they learn. Fig. 7 shows that both the number of nodes and depth first increased and then eventually became stable, which implies that this process is a structure convergence process under optimization.

It is through the information in Table 6 that the structure of the training process is made clear. The high initial SAR values, for which

the system continually utilizes the wide regions of the sample space. This is not out of the ordinary for early policy learning. During the episodes the SAR values also decreased, thus meaning the agents started to generate steady and reusable decision hierarchies. It appears that the structure evolution dynamics may tend to favor optimal shapes, rather than to develop out-of-control. The reduction tendency of SAR patterns in Fig. 8, and also agents took fewer structural actions once the policies were well-developed.

Table 7 addresses inter-agent cooperation. The MACS increased from 0.51 to 0.78. It means that the agents learned not only good individual behaviors but also how to properly act synergistically over time. We established that the improvements were made through distributed execution, and implementation of algorithm without central controller. Thus we provided the concept of centralized training-decentralized execution. The transport of objects during handoff and cooperative navigation tasks is successful for the higher MACS scores, as the exact timing and spatial correlation are necessary for the former. The trend shown in Fig. 9 validate that MACS consistently improves with the development of cooperative strategies.

To give a clearer picture of generality, we further inspected the effect of varying the number of agents (3, 5, and 8) and obstacle

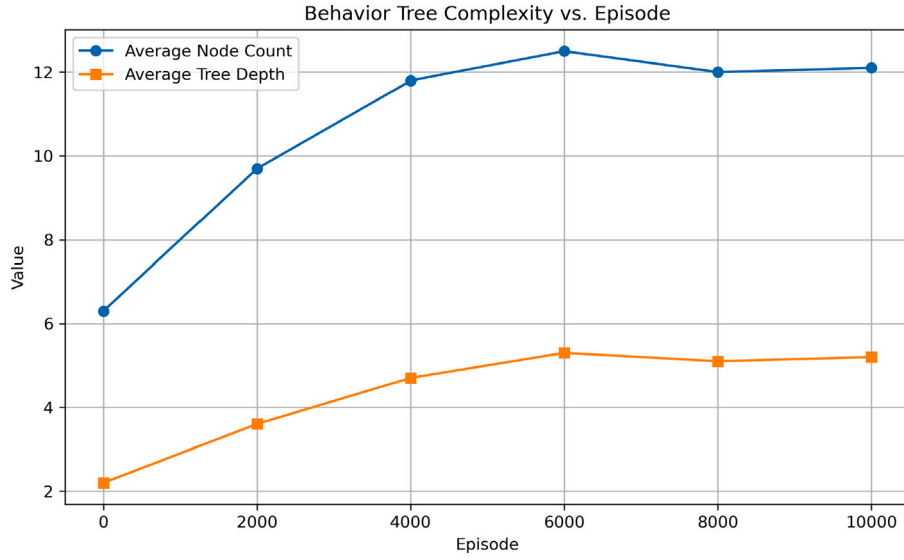


Fig. 7. Evolution of behavior tree complexity over episodes in the task allocation scenario.

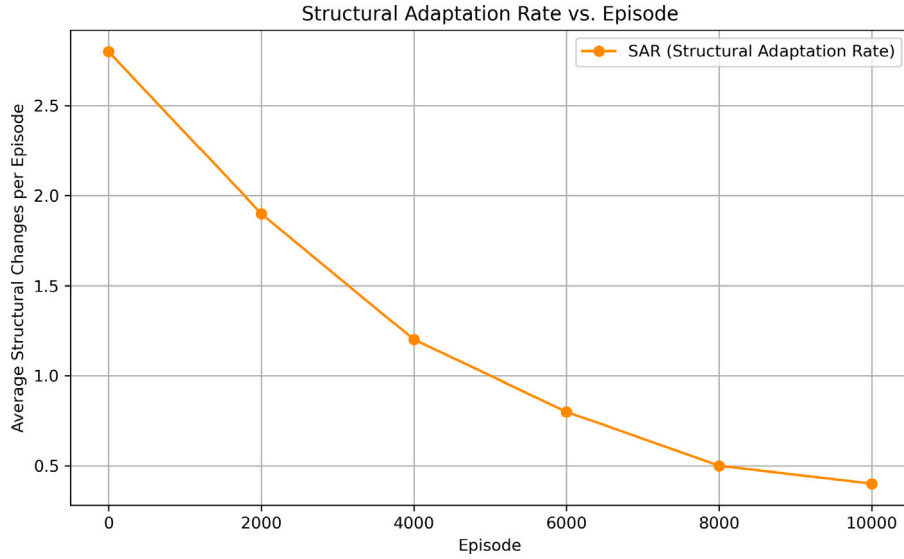


Fig. 8. Structural adaptation rate over training episodes.

densities (10%–30%) that are already summarized in the simulation setup (Table 1). Increasing the team size from 3 to 8 agents resulted in less than a 3% drop in TSR and a moderate increase in DL (on the order of one to two extra decision steps), indicating that the learned BT hierarchies remain stable under denser interactions. Similarly, higher obstacle densities primarily affected DL rather than TSR, suggesting that structural evolution helps preserve robustness in success while trading off a small amount of latency. We note that all reported scenarios are variations of the same family of navigation and cooperation tasks; systematic evaluation on completely unseen task types is left as an important direction for future work.

Although our experiments focus on ROS2/Gazebo cooperative navigation and task allocation, the proposed MAPPO–BTE mechanism is in principle domain-agnostic and has the potential to transfer to broader multi-agent settings such as heterogeneous robot fleets (manipulation + mobile platforms), warehouse coordination with task preemption, multi-UAV patrolling under partial observability, and human-robot teaming with safety constraints. In such scenarios, the dual-policy design (action + structure) could preserve BT interpretability while supporting online structural adaptation under a centralized-training/decentralized-execution regime.

4.2. Comparison with baselines

The performance of our proposed MAPPO + Dynamic BT framework was further validated by comparison with two baseline methods: (i) a static BT integrated with MAPPO and (ii) a dynamic BT trained using a simpler PPO framework. As shown in Table 8, our model outperforms both baselines across all metrics. These baselines were selected following prevailing trends in the literature. For example, Li et al. (2021) proposed the MDRL-BT framework, which embeds deep RL algorithms such as PPO and SAC into BT nodes via an options-based design, enabling local learning within BT structures but remaining limited to single-agent, non-evolving scenarios. Likewise, the MARL-BT framework (Li et al., 2024) combines static BTs with MARL to scale to multi-agent settings but lacks inherent BT adaptation. In contrast, our MAPPO–BTE approach bridges these lines by enabling intelligent structural adaptation and coordinated BT evolution during decentralized multi-agent training, yielding significant improvements across all performance measures.

The highest performance was reached with the proposed system with the largest TSR: $91.3\% \pm 2.1\%$, and it was significantly superior

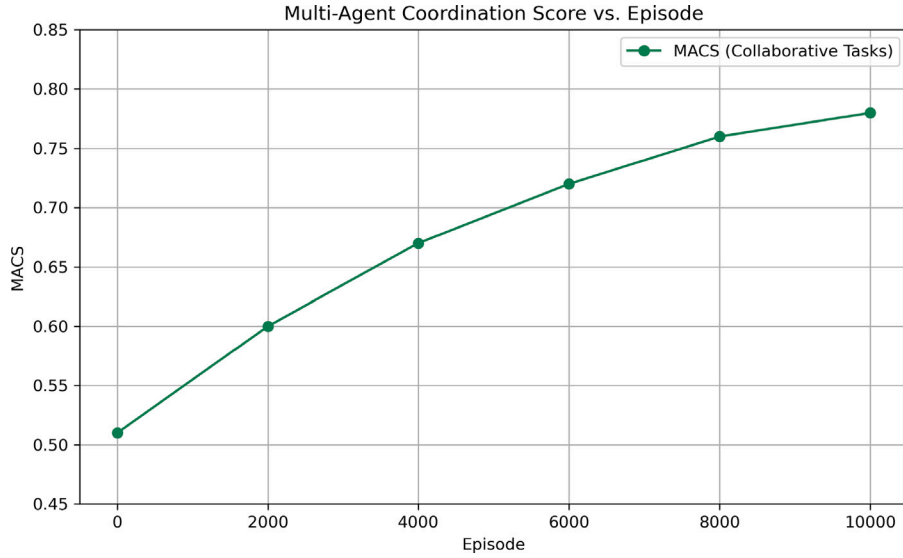


Fig. 9. Improvement in multi-agent coordination score (MACS) over time in collaborative task scenarios.

Table 8

Comparative performance analysis of the proposed and baseline methods.

Metric	Static BT+MAPPO	Dynamic BT+PPO	Proposed method (MAPPO+Dynamic BT)
Task success rate	78.6 $\pm$ 3.4%	84.1 $\pm$ 2.7%	91.3 $\pm$ 2.1%
Decision latency	18.7 $\pm$ 1.2	15.9 $\pm$ 1.1	11.6 $\pm$ 0.9
BT node count	9.2 $\pm$ 2.1	17.6 $\pm$ 2.5	14.3 $\pm$ 1.8
Avg. tree depth	3.1 $\pm$ 0.5	6.2 $\pm$ 0.7	5.6 $\pm$ 0.4
MACS (collaborative)	0.63 $\pm$ 0.09	0.71 $\pm$ 0.08	0.78 $\pm$ 0.06

with respect to the baselines. To this end, it also led to a spectacular reduction of the decision latency, which was reduced to 11.6 ± 0.9 steps. The PPO based BT architecture exhibited more nodes of 17.6, but it did not produce superior results as the hierarchical evolutionary and centralized training were required for it.

In collaborative situations, the MACS score shows that our method takes better advantage of the coordination capabilities. Agents learnt via our approach achieved a final MACS score of 0.78 which displayed better temporal and spatial coherence than the baseline methods. The hybrid MAPPO and BT evolution system not only provides improved individual performance, but also new cooperative behaviors for multi-agent systems.

As shown in Table 9, direct numerical comparison with BT with GP-based evolution (Iovino et al., 2023), MDRL-BT (Li et al., 2021), and MARL-BT (Li et al., 2024) is not feasible due to differences in domains and evaluation protocols. BT with GP-based evolution employs genetic programming for BT evolution outside training and is limited to single-agent benchmarks with conventional reward/step metrics. MDRL-BT integrates deep RL algorithms such as PPO and SAC within BT nodes through an options-based framework, enabling local policy learning but remaining restricted to single-agent, non-evolving settings. MARL-BT supports multi-agent training but relies on fixed, static BTs without structural adaptation. In contrast, our framework enables on-policy structural evolution during RL training under a decentralized MAPPO setting. Furthermore, our evaluation goes beyond the standard TSR by incorporating DL, BTC, SAR, and MACS, which specifically capture adaptability–interpretability trade-offs in evolving multi-agent behavior trees. This simultaneous adaptation of both policy and BT hierarchy during training constitutes the unique contribution of our work: unlike prior approaches, it scales naturally to multi-agent domains, supports continuous online adaptation rather than offline evolution, and empirically preserves stability under Centralized Training with Decentralized Execution (CTDE). By bridging scalability, adaptability, and interpretability, our method fills a gap that existing frameworks leave unaddressed.

From an algorithmic standpoint, the key difference to existing BT-based approaches is that structural evolution is performed *on-line and on-policy* within a CTDE MAPPO loop. BT with GP-based evolution (Iovino et al., 2023) evolves trees offline via genetic programming and then freezes them before deployment; MDRL-BT (Li et al., 2021) learns local RL options inside a fixed BT skeleton; MARL-BT (Li et al., 2024) scales static BTs to multi-agent settings. In contrast, MAPPO–BTE updates both the policy parameters and the BT structure jointly during training: Insert/Prune/Reorder operations are triggered by advantage- and reward-based signals and are subject to stability checks, so that BT hierarchies co-evolve with the MARL policy. This online, CTDE-compatible structural adaptation is not present in prior BT with GP-based evolution, MDRL-BT, or MARL-BT formulations and constitutes the core novelty of the proposed framework.

We acknowledge that stronger hierarchical MARL methods such as Hierarchical Actor-Critic (Nachum et al., 2018), Options-PPO (Bacon et al., 2017), or intent-sharing MAPPO (Iqbal and Sha, 2019) could also serve as competitive baselines. However, these approaches mainly address hierarchical skill discovery or communication protocols, whereas our focus lies in the structural evolution of BTs within a MARL setting. For this reason, we deliberately selected baselines (Static BT+MAPPO and Dynamic BT+PPO) that isolate the effect of BT adaptation, ensuring that the contribution of structural evolution is evaluated in a controlled and interpretable manner.

4.3. Time consumption and complexity analysis

The proposed MAPPO–BTE framework introduces additional BT evolution operations on top of standard MAPPO training. From an algorithmic perspective, the overall time complexity remains comparable to conventional MAPPO: policy optimization requires $O(NT)$ operations per update (N agents, T rollout length), while BT traversal and modification add at most $O(k)$, where k is the current node count of the tree. Since $k \ll NT$ in our experiments, the asymptotic complexity is not altered, and the training process scales in the same order as MAPPO.

Table 9

Qualitative comparison of our framework with closely related approaches from the literature.

Method	Domain	BT structural evolution	Multi-agent
BT with GP-based evolution (Iovino et al., 2023)	Single-agent RL	Yes (offline via GP)	No
MDRL-BT (Li et al., 2021)	Single-agent RL	No (static BT with RL nodes)	No
MARL-BT (Li et al., 2024)	Multi-agent RL	No (static BT)	Yes
Proposed method	Multi-agent RL	Yes (during RL training)	Yes

Table 10

Relative time consumption and complexity of compared methods.

Method	Training complexity	Extra overhead	Inference latency
Static BT+MAPPO	$O(NT)$	None	$O(d)$, $d \approx 3$
Dynamic BT+PPO	$O(NT)$	Moderate (tree growth)	$O(d)$, $d \approx 6$
Proposed method	$O(NT + k)$	$\sim 7\text{--}10\%$	$O(d)$, $d \approx 5$

In terms of practical training cost, BT structural updates (insert, prune, reorder) contribute a small overhead per episode compared to Static BT+MAPPO. During inference, the decision-making latency is dominated by BT traversal, with average depth of $d \leq 6$, resulting in sub-millisecond decision times per agent. Table 10 summarizes the comparative training and inference costs.

This confirms that the proposed framework preserves the scalability of MAPPO while introducing only a negligible overhead, making it suitable for real-time multi-agent robotic applications.

5. Conclusion

In this study, we provided a novel hybrid approach that combines the MAPPO with dynamic BT evolution in enhancing decentralized decision-making systems for multi-agent systems. BTs have a modular nature, which allows the explanation of their decisions, but their rigid structure makes them unsuitable in dynamically changing scenarios. During training, BT structures can grow independently and, thus, agents can build their decision trees over learned policies without any centralized policy, control, or tasks written manually.

Experimental tests on three typical testing scenarios showed that the system provided sustained improvements in the testing performances. Under the task allocation scenario, TSR was 91.3% when the system obtained 36% DL reduction over 10,000 episodes. The BTC value demonstrated that the growth of the number of nodes and the depth of the tree were leveled out over time, which suggested that the structural convergence was achieved successfully. The agents converged to a stable SAR with values ranging from 2.8 to 0.4; they learned to be near-optimal in BTs through minimal exploration. A significant increase was observed for MACS in collaborative situations from 0.51 to 0.78, which reflected the emergence of synchronized behavior.

The proposed approach allows policies to change their decision tree structure in a structural way. The relationship between learning and modular representations of decisions enables us to link symbolic and sub-symbolic models of decision making. The application of MAPPO enables effective and robust multi-agent training with MAPPO, while the decentralized BT execution allows for real-time operation as well as scalability. This study is the first systematic study of MARL with BT evolution in ROS2-compatible simulation environments.

While the proposed framework demonstrates promising results in simulated multi-agent settings, its applicability to heterogeneous or physical robotic systems remains to be validated. Future work will focus on extending the evaluation to heterogeneous agents and real-world robotic platforms to further assess generalization under uncertainty, sensor noise, and limited computational resources.

CRedit authorship contribution statement

Koray Ozdemir: Software, Methodology. **Adem Tuncer:** Validation, Software, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This study was supported by the Scientific and Technological Research Council of Türkiye (TÜBİTAK) under Grant Number 124E441. The authors would like to thank TÜBİTAK for its support.

Appendix

This appendix provides the complete mathematical details of the proposed MAPPO–BTE framework. For clarity and readability, the main manuscript (Sections 3–4) presented only the essential objectives and high-level descriptions. Here, we include the full derivations, probability functions, and auxiliary formulations referenced throughout the paper.

A.1. Detailed formulations for BT evolution mechanism

Structural transformation function. Given the current state s_t , the agent selects both an action a_t and a potential BT modification T according to:

$$BT_{t+1} = T(BT_t, \pi_\theta(s_t)) \quad (10)$$

where T is a probabilistic transformation operator. The decision to apply a modification is controlled by the probability function:

$$P(T) = \frac{1}{1 + e^{-\lambda(\delta - \tau)}} \quad (11)$$

with:

- λ = scaling factor,
- δ = confidence in modification effectiveness,
- τ = threshold controlling modification frequency.

Reward function. The reward guiding BT updates is a weighted sum of success, efficiency, and adaptability:

$$R(BT) = w_s R_s + w_e R_e + w_a R_a \quad (12)$$

where:

$$\begin{aligned} R_s &= \frac{\text{Successful Executions}}{\text{Total Attempts}}, \\ R_e &= 1 - \frac{\text{Average Execution Time}}{\text{Maximum Allowed Time}}, \\ R_a &= \frac{\text{Performance in Unseen Scenarios}}{\text{Baseline Performance}}. \end{aligned} \quad (13)$$

The weights (w_s, w_e, w_a) adapt dynamically to favor exploration early in training and stability later.

Exploration–exploitation balance. Exploration probability decays over time:

$$P_{\text{explore}}(t) = P_0 e^{-\beta t} \quad (14)$$

where P_0 is the initial exploration rate and β is the decay factor.

Final selection. Among candidate modifications $\{T_1, \dots, T_n\}$, the one with maximum expected reward improvement is selected:

$$T^* = \arg \max_{T_i} \Delta R(BT_i) \quad (15)$$

A.2. Detailed formulations for MAPPO integration

Dual-policy formulation. Each agent follows:

$$a_t \sim \pi_a(a_t | s_t, BT_t), \quad T_t \sim \pi_T(T_t | s_t, BT_t) \quad (16)$$

If $T_t = \emptyset$, the BT remains unchanged; otherwise, $BT_{t+1} = T(BT_t, \pi_T(s_t))$.

Confidence update. The structural confidence metric δ is updated as:

$$\delta \leftarrow \delta + \eta R_T(BT_t) \quad (17)$$

where η is the update step size and $R_T(BT_t)$ is the local reward from transformation.

Regularization. The transformation policy loss incorporates entropy regularization:

$$L_T = -\mathbb{E}_{T_t \sim \pi_T} [R_T(BT_t) - \alpha H(\pi_T)] \quad (18)$$

where $H(\pi_T)$ is entropy and α controls exploration.

A.3. Structural optimization of BTs

Pruning metric. The activation score of a node n is:

$$A(n) = \frac{\sum_{t=1}^T \mathbb{I}(n_t \text{ executed}) \cdot R(n_t)}{T} \quad (19)$$

A node is pruned if $A(n) < \theta_p$.

Depth control. Tree depth is limited to $D_{\max}$. If exceeded, cumulative activation scores of subtrees are computed:

$$A(T_i) = \sum_{n \in T_i} A(n) \quad (20)$$

The least useful subtree is merged or replaced to keep BTs compact.

Execution efficiency reward. Structural optimization also considers:

$$R_e = 1 - \frac{\text{Average Execution Time of BT}}{\text{Maximum Allowed Time}} \quad (21)$$

rewarding agents for faster decision-making.

A.4. Policy optimization in MAPPO

Trust region constraint.

$$L(\theta) = \mathbb{E}[\min(r_t A_t, \bar{r}_t A_t)], \quad (22a)$$

$$r_t := \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}, \quad (22b)$$

$$\bar{r}_t := \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon). \quad (22c)$$

Centralized value function. The critic estimates:

$$V(s_t) = \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k} \mid s_t \right] \quad (23)$$

where s_t is the joint state of all agents.

Adaptive learning rate. The policy learning rate adapts to entropy:

$$\alpha_t = \alpha_0 \cdot e^{-\beta H(\pi_t)} \quad (24)$$

with α_0 the initial rate and β a decay factor.

Data availability

Data will be made available on request.

References

- Bacon, P.-L., Harb, J., Precup, D., 2017. The option-critic architecture. In: Proceedings of the AAAI Conference on Artificial Intelligence.
- Busoniu, L., Babužić-Fischer, A., De Schutter, B., Ernst, D., 2010. Multi-agent reinforcement learning: An overview. Innovations in multi-agent systems and applications-1 33, 183–221.
- Colledanchise, M., Ögren, P., 2018. Behavior Trees in Robotics and AI: An Introduction. CRC Press.
- Garnelo, M., Shanahan, M., 2019. Reconciling deep learning with symbolic artificial intelligence: Representing objects and relations. Curr. Opin. Behav. Sci. 29, 17–23.
- Guo, Y., Liu, J., Yu, R., Hang, P., Sun, J., 2025. MAPPO-PIS: A multi-agent proximal policy optimization method with prior intent sharing for CAVs' cooperative decision-making. In: Computer Vision – ECCV 2024 Workshops. Springer, pp. 244–263.
- Haarnoja, T., Zhou, A., Abbeel, P., Levine, S., 2018. Soft actor-critic algorithms and applications. arXiv preprint arXiv:1812.05905.
- Hernandez-Leal, P., Kartal, B., Taylor, M.E., 2019. A survey and critique of multi-agent deep reinforcement learning. Auton. Agents Multi Agent Syst. 33 (6), 750–797.
- Iovino, M., Styrd, J., Falco, P., Smith, C., 2023. A framework for learning behavior trees in collaborative robotic applications. In: 2023 IEEE 19th International Conference on Automation Science and Engineering (CASE). pp. 1–8.
- Iqbal, S., Sha, F., 2019. Actor-attention-critic for multi-agent reinforcement learning. In: International Conference on Machine Learning. PMLR, pp. 2961–2970.
- Jia, Z., Li, J., Qu, X., Wang, J., 2025. Enhancing multi-agent systems via reinforcement learning with LLM-based planner and graph-based policy. arXiv preprint arXiv:2503.10049.
- Kaelbling, L.P., Littman, M.L., Moore, A.W., 1996. Reinforcement learning: A survey. J. Artificial Intelligence Res. 4, 237–285.
- Lee, D., Yannakakis, M., 2002. Principles and methods of testing finite state machines-a survey. Proc. IEEE 84 (8), 1090–1123.
- Li, X., Li, Y., Zhang, J., Xu, X., Liu, D., 2024. Embedding multi-agent reinforcement learning into behavior trees with unexpected interruptions. Complex Intell. Syst. 10, 3273–3282.
- Li, L., Wang, L., Li, Y., Sheng, J., 2021. Mixed deep reinforcement learning-behavior tree for intelligent agents design. In: ICAART (1). pp. 113–124.
- Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P., Mordatch, I., 2017. Multi-agent actor-critic for mixed cooperative-competitive environments. In: Advances in Neural Information Processing Systems. NeurIPS.
- Marzinotto, A., Colledanchise, M., Smith, C., Ögren, P., 2014. Towards a unified behavior trees framework for robot control. In: Proceedings of the IEEE International Conference on Robotics and Automation. ICRA, pp. 5420–5427.
- Nachum, O., Gu, S., Lee, H., Levine, S., 2018. Data-efficient hierarchical reinforcement learning. In: Advances in Neural Information Processing Systems.
- Ögren, P., 2012. Increasing modularity of UAV control systems using behavior trees. In: Aiaa guidance, navigation, and control conference. ICRA, p. 4458.
- Ozdemir, K., Tuncer, A., 2025. Navigation of autonomous mobile robots in dynamic unknown environments based on dueling double deep q networks. Eng. Appl. Artif. Intell. 139, 109498.
- Russell, S., Norvig, P., 2010. Artificial Intelligence: A Modern Approach. Pearson Education.
- Sadhukhan, P., Selmic, R.R., 2021. Multi-agent formation control with obstacle avoidance using proximal policy optimization. In: 2021 IEEE International Conference on Systems, Man, and Cybernetics. SMC, IEEE, pp. 2694–2699.
- Scherf, L., Schmidt, A., Pal, S., Koert, D., 2023. Interactively learning behavior trees from imperfect human demonstrations. Front. Robot. AI 10, 1152595.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O., 2017. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347.
- Sutton, R.S., Barto, A.G., 2018. Reinforcement Learning: An Introduction, second ed. MIT Press.
- Zhang, Y., Li, M., Zeng, X., Song, N., Zhang, J., Peng, B., Zhang, P., 2024. Research on multi-agent cooperative tasks based on improved proximal policy optimization. In: China Conference on Command and Control. Springer, pp. 230–244.
- Zhang, K., Yang, Z., Başar, T., 2021. Multi-agent reinforcement learning: A selective overview of theories and algorithms. Handb. Reinf. Learn. Control. 321–384.
- Zhao, C., Deng, C., Liu, Z., Zhang, J., Wu, Y., Wang, Y., Yi, X., 2023. Interpretable reinforcement learning of behavior trees. In: Proceedings of the 2023 15th International Conference on Machine Learning and Computing. pp. 492–499.